Python Case Studies

Easy explanations, Python source code, and sample outputs

Note: The programs use simple Python concepts such as variables, strings, lists, tuples, loops, and conditional statements. Sample values are included for practice and can be changed.

Advanced String-Based Registration Record Analyzer

Explanation: Checks registration records, extracts the year and branch, and identifies duplicate or invalid records.

Python code

```
records = ["2026-CSE-101", "2026-DS-102", "2026-CSE-101", "26-IT-103"]
seen = []

for code in records:
    parts = code.split("-")
    if len(parts) == 3 and len(parts[0]) == 4 and parts[0].isdigit() and parts[2].isdigit():
        if code in seen:
            print(code, "is a duplicate")
        else:
            print(code, "is valid")
            print("Year:", parts[0], "Branch:", parts[1])
            seen.append(code)
    else:
        print(code, "is invalid")

print("Unique valid records:", len(seen))
```

Sample output

```
2026-CSE-101 is valid
Year: 2026 Branch: CSE
2026-DS-102 is valid
Year: 2026 Branch: DS
2026-CSE-101 is a duplicate
26-IT-103 is invalid
Unique valid records: 2
```

How it works: split() separates a registration code into parts. Conditions check the format. The list seen helps identify duplicate codes.

Multi-Event Sports Scheduling and Scoring

Explanation: Displays sports event timings, checks for overlapping events, and finds the team with the highest score.

Python code

```
events = [("Running", 9, 10), ("Jumping", 10, 11), ("Chess", 12, 13)]
teams = [("Team A", 15), ("Team B", 20), ("Team C", 18)]

print("Event Schedule:")
for event in events:
    print(event[0], "from", event[1], "to", event[2])

print("Schedule clashes:")
for i in range(len(events)):
    for j in range(i + 1, len(events)):
        if events[i][1] < events[j][2] and events[j][1] < events[i][2]:
            print(events[i][0], "and", events[j][0], "have a clash")

winner = teams[0]
for team in teams:
    if team[1] > winner[1]:
        winner = team

print("Winner:", winner[0], "with", winner[1], "points")
```

Sample output

```
Event Schedule:
Running from 9 to 10
Jumping from 10 to 11
Chess from 12 to 13
```

```
Schedule clashes:
Winner: Team B with 20 points
```

How it works: Each event is stored in a tuple containing its name, start time, and end time. Loops compare timings and find the highest team score.

Smart Public Transport Fare Reconciliation

Explanation: Calculates the expected fare collection and compares it with the amount actually collected.

Python code

```
passengers = [("Adult", 30), ("Student", 15), ("Senior", 20)]
collected = 65
expected = 0

for category, fare in passengers:
    print(category, "fare:", fare)
    expected = expected + fare

difference = collected - expected
print("Expected revenue:", expected)
print("Collected revenue:", collected)

if difference == 0:
    print("Revenue matches")
elif difference > 0:
    print("Extra amount:", difference)
else:
    print("Short amount:", -difference)
```

Sample output

```
Adult fare: 30
Student fare: 15
Senior fare: 20
Expected revenue: 65
Collected revenue: 65
Revenue matches
```

How it works: The loop adds all passenger fares. The difference tells us whether the collected amount matches the expected revenue.

Disaster Relief Supply Allocation

Explanation: Distributes available supplies to different areas and calculates any shortage.

Python code

```
areas = [("Area A", 40), ("Area B", 30), ("Area C", 25)]
stock = 70

for name, demand in areas:
    if stock >= demand:
        given = demand
    else:
        given = stock

    stock = stock - given
    shortage = demand - given
    print(name, "received:", given, "Shortage:", shortage)

print("Remaining stock:", stock)
```

Sample output

```
Area A received: 40 Shortage: 0
Area B received: 30 Shortage: 0
Area C received: 0 Shortage: 25
Remaining stock: 0
```

How it works: The program supplies areas in list order. If enough stock is available, the full demand is given. Otherwise, the remaining stock is supplied.

Integrated Manufacturing Production Planner

Explanation: Checks whether enough raw material is available and calculates good and defective products.

Python code

```
raw_material = 100
material_per_item = 2
order_quantity = 40
defective = 3

needed = order_quantity * material_per_item

if raw_material >= needed:
    produced = order_quantity
else:
    produced = raw_material // material_per_item

good_items = produced - defective
if good_items < 0:
    good_items = 0

print("Items produced:", produced)
print("Defective items:", defective)
print("Good items:", good_items)
print("Material left:", raw_material - produced * material_per_item)

if good_items >= order_quantity:
    print("Order completed")
else:
    print("Order not completed")
```

Sample output

```
Items produced: 40
Defective items: 3
Good items: 37
Material left: 20
Order not completed
```

How it works: The program calculates the material required for an order. It subtracts defective items from the total produced to find good items.

Multi-Criteria Library Resource Distribution

Explanation: Shares available book copies among library branches and reports how many more copies are required.

Python code

```
branches = [("Branch A", 5), ("Branch B", 4), ("Branch C", 3)]
books_available = 9

for name, required in branches:
    if books_available >= required:
        given = required
    else:
        given = books_available

    books_available = books_available - given
    needed = required - given
    print(name, "received:", given, "Still needed:", needed)

print("Copies remaining:", books_available)
```

Sample output

```
Branch A received: 5 Still needed: 0
Branch B received: 4 Still needed: 0
Branch C received: 0 Still needed: 3
Copies remaining: 0
```

How it works: The loop distributes the available books to each branch. It updates the stock and calculates the number of copies still needed.

Smart Campus Incident and Maintenance Prioritization

Explanation: Calculates a priority score for maintenance reports and classifies each issue.

Python code

```
reports = [("Broken light", 2, 3), ("Water leakage", 5, 2), ("Damaged desk", 1, 1)]
# Each record contains issue, severity, and number of reports

for issue, severity, count in reports:
    score = severity * count
    print(issue, "Priority score:", score)

    if score >= 8:
        print("Priority: High")
    elif score >= 3:
        print("Priority: Medium")
    else:
        print("Priority: Low")
```

Sample output

```
Broken light Priority score: 6
Priority: Medium
Water leakage Priority score: 10
Priority: High
Damaged desk Priority score: 1
Priority: Low
```

How it works: The priority score is calculated by multiplying severity by the number of reports. Conditions classify the priority as High, Medium, or Low.

University Academic Progression Audit

Explanation: Checks subject marks, attendance, and prerequisite completion to decide whether a student can progress.

Python code

```
marks = [65, 42, 70]
attendance = 80
prerequisite_completed = True
pass_mark = 40
eligible = True

for mark in marks:
    if mark < pass_mark:
        eligible = False

if attendance < 75:
    eligible = False

if prerequisite_completed == False:
    eligible = False

if eligible:
    print("Student is eligible for next semester")
else:
    print("Student is not eligible")
    if attendance < 75:
        print("Reason: Low attendance")
    if prerequisite_completed == False:
        print("Reason: Prerequisite not completed")
    for mark in marks:
        if mark < pass_mark:
            print("Reason: Failed subject with mark", mark)
```

Sample output

```
Student is eligible for next semester
```

How it works: The loop checks every subject mark. The conditions also check attendance and prerequisite completion before deciding eligibility.

Integrated E-Commerce Order Processing and Reconciliation

Explanation: Processes product orders, updates stock after returns, and calculates total sales revenue.

Python code

```
stock = [10, 8, 5]
prices = [100, 200, 150]
ordered = [2, 3, 2]
returned = [1, 0, 1]
```

```
revenue = 0

for i in range(len(stock)):
    dispatched = ordered[i] - returned[i]
    stock[i] = stock[i] - dispatched
    revenue = revenue + dispatched * prices[i]

    print("Product", i + 1)
    print("Dispatched:", dispatched)
    print("Returned:", returned[i])
    print("Stock left:", stock[i])

print("Total revenue:", revenue)
```

Sample output

```
Product 1
Dispatched: 1
Returned: 1
Stock left: 9
Product 2
Dispatched: 3
Returned: 0
Stock left: 5
Product 3
Dispatched: 1
Returned: 1
Stock left: 4
Total revenue: 850
```

How it works: The lists store stock, prices, ordered quantities, and returned quantities. The loop calculates dispatched items, updates stock, and adds revenue.

Comprehensive Smart Campus Management Simulation

Explanation: Uses a simple menu to display student details, attendance, marks, and event registration.

Python code

```
name = "Anu"
attendance = 85
marks = [78, 82, 90]
events = []

while True:
    print("\n1. Student details")
    print("2. Attendance")
    print("3. Marks")
    print("4. Register for event")
    print("5. Exit")

    choice = input("Enter choice: ")

    if choice == "1":
        print("Student name:", name)
    elif choice == "2":
        print("Attendance:", attendance, "%")
    elif choice == "3":
        print("Marks:", marks)
        print("Total:", sum(marks))
    elif choice == "4":
        event = input("Enter event name: ")
        events.append(event)
        print("Registration successful")
    elif choice == "5":
        print("Registered events:", events)
        print("Thank you")
        break
    else:
        print("Invalid choice")
```

Sample output

```
Example run:
Enter choice: 1
Student name: Anu
Enter choice: 5
Registered events: []
Thank you
```

How it works: The while loop repeatedly displays the menu. The if-elif statements perform the selected operation. The break statement exits the program.